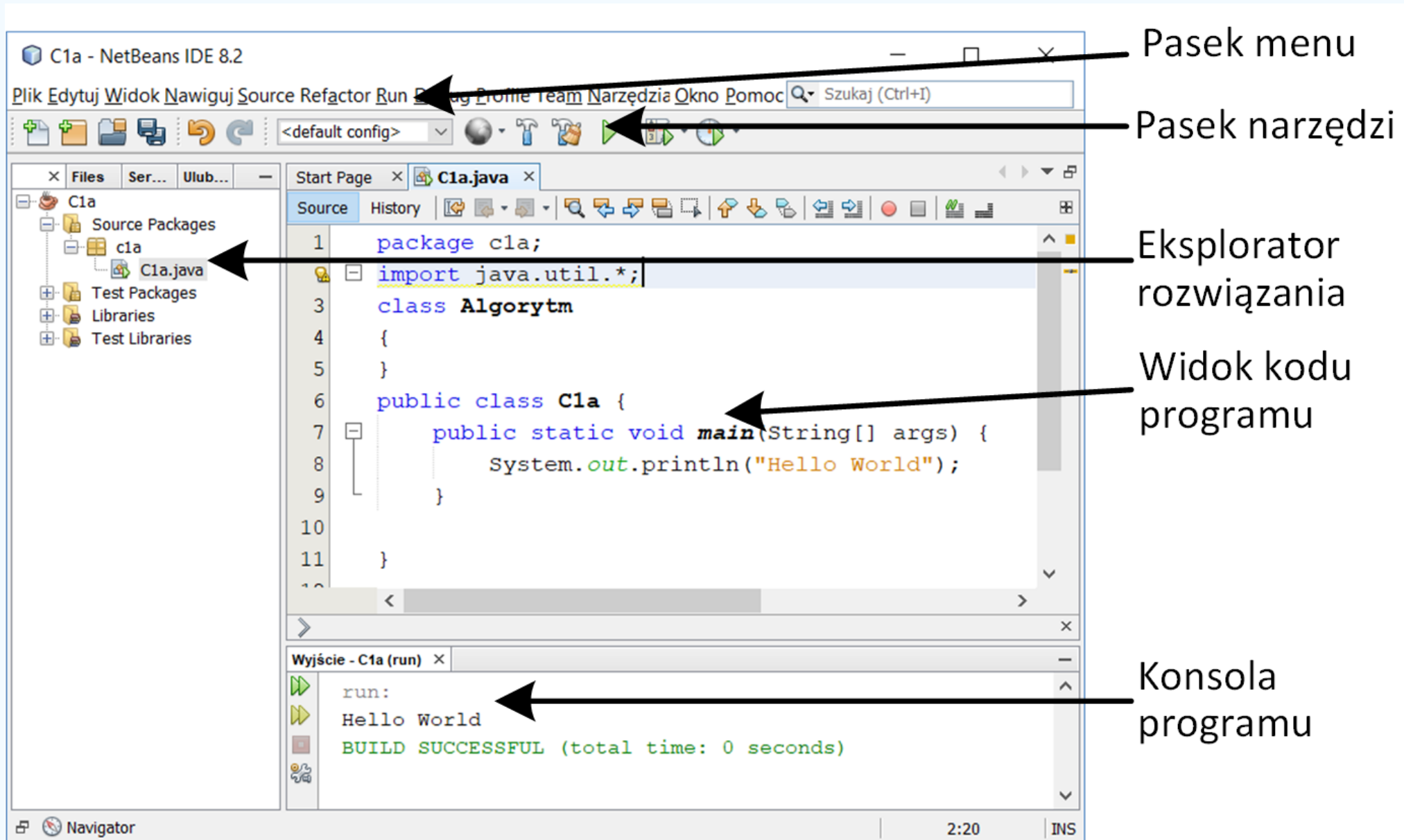


Programowanie w języku Java

Zbigniew Marszalek

Podstawowa struktura programu



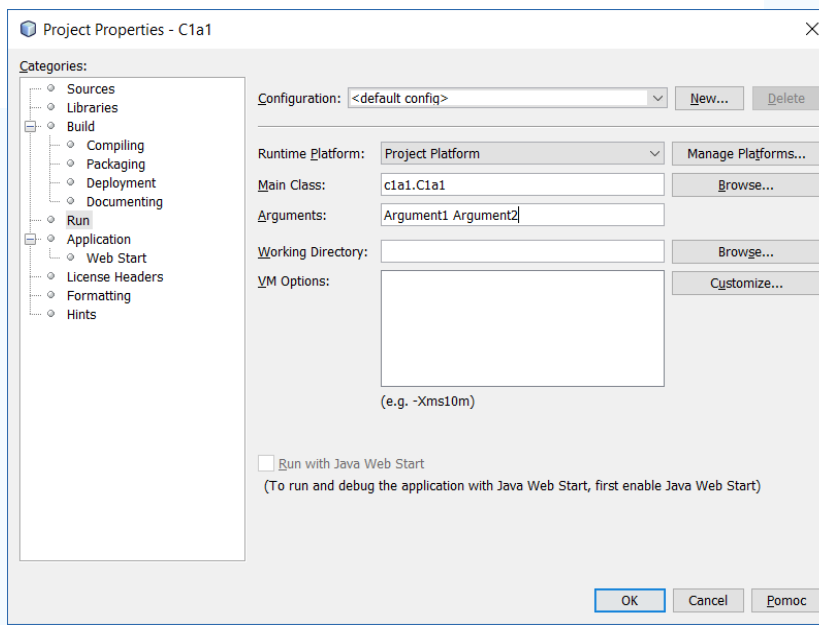
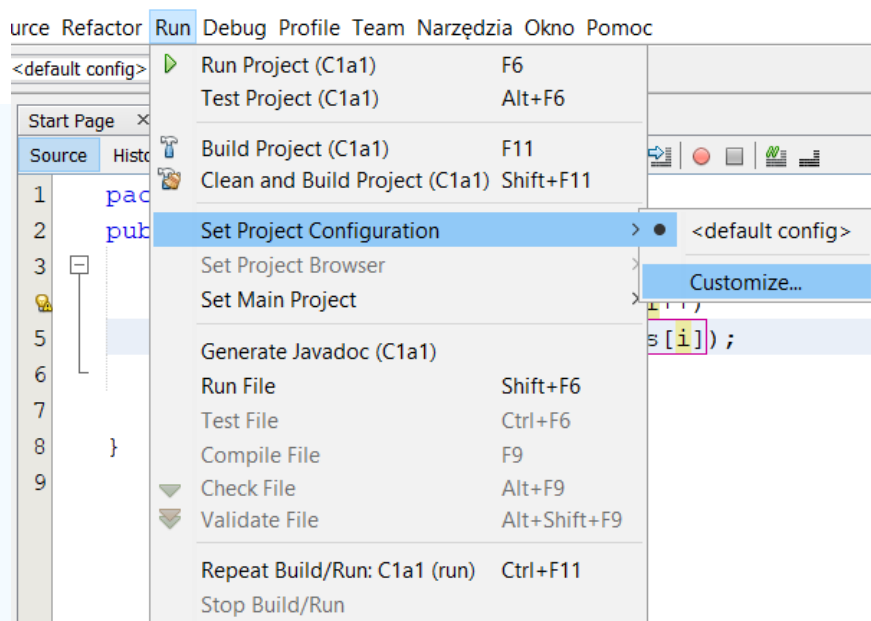
Struktura programu

- Instrukcja import określa wykorzystywaną przez program przestrzeń bibliotek języka Java. Biblioteki klas pogrupowane są w ramach przestrzeni nazw System i zawierają wszystkie proste typy danych.
- Cały program i wszystkie wykorzystywane dane muszą się znajdować w ramach definicji typów. Algorytmy i przetwarzane dane należy osadzać w klasie, strukturze, typie wyliczeniowym lub interfejsie.
- Metoda Main() jest punktem wejścia do programu i musi zostać umieszczona w programie

Otrzymywane argumenty na wejściu

Statyczna metoda Main() może otrzymywać na wejściu argumenty i zwracać wynik całkowito liczbowy.

```
package c1a1;  
public class C1a1 {  
    public static void main(String[] args) {  
        for(int i=0;i<args.length;i++)  
            System.out.println(args[i]);  
    }  
}
```



```
run:  
Argument1  
Argument2
```

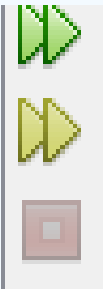
Typy podstawowe

Typ podstawowe	Opis
boolean	Typ logiczny przyjmujący dwie wartości
byte	8 – bitowy typ znakowy (od -128 do 127)
short	16 – bitowa liczba całkowita ze znakiem
int	31 – bitowa liczba całkowita ze znakiem
long	64 – bitowa liczba całkowita ze znakiem
float	32 – bitowa liczba rzeczywista
double	64 – bitowa liczba rzeczywista
char	Znak w standardzie Unicode

Typ logiczny - boolean

W języku Java zdefiniowano typ logiczny o nazwie boolean. Zmienne logiczne przyjmuje wartości true albo false. Nie ma możliwości rzutowania zmiennych całkowito liczbowych na zmienne logiczne ani zmiennych logicznych na liczby. Typ boolean jest wymagany w wyrażeniach warunkowych oraz w strukturach sterujących takich jak if i for. Przykład użycia:

```
boolean b = true;  
System.out.println(b) ;  
b = false;  
System.out.println(b) ;
```



```
run:  
true  
false
```

Typ byte

Typ `byte` reprezentuje liczbę całkowitą ze znakiem zapamiętaną na jednym bajcie (tzn. od `-128` do `127`). Zmienne typu `byte` są przydatne przy przetwarzaniu strumieni danych binarnych z pliku lub otrzymywanych przez sieć.

```
byte a = 127; byte b = -128;  
System.out.println("a = " + a + " b = " + b);
```



run:



a = 127 b = -128

Typ short, int i long

Typy short, int i long reprezentują odpowiednio liczby całkowite ze znakiem zapisane na dwóch, czterech i ośmiu bajtach. Na liczbach typu short, int i long możemy dokonywać operacji bitowych np. przesunięcia o jeden bit w lewo.

```
short c1 = 0x4f00;  
System.out.printf("%d %x%n", c1, c1);  
int c2 = 0x4f000000;  
System.out.printf("%d %x%n", c2, c2);  
c2 = c2<<1;  
System.out.printf("%d %x%n", c2, c2);
```



```
run:
20224 4f00
1325400064 4f0000000
-1644167168 9e0000000
```

W programie dokonano przesunięcia w lewo liczby całkowitej mającej ustawiony przed ostatni bit na jeden. Wyniku przesunięcia na pozycji najbardziej znaczącej (pozycja znaku liczby) znalazła się wartość jeden. Liczba została odczytana jako liczba ujemna zapisana w kodzie uzupełnienia do dwa, co znalazło odzwierciedlenie na wydruku.

Typ float i double

Typy float i double są wykorzystywane do obliczeń numerycznych i są zapamiętane w postaci wykładniczej. Typ float mają dokładność sześciu cyfr mantysy a liczby typu double podwójnej precyzji 15 cyfr mantysy. Operacje na liczbach zmiennoprzecinkowych zwracają wartość $\pm$ nieskończoność (ang. $\pm$ Infinity) dla działań których wynik jest nie określony np. dzielenie przez zero.

```
float a = 256.67f;  
double b = 2.1e21;  
System.out.printf("%g %g%n", a, b);
```



run:



256,670 2,10000e+21

Typ char

Typ char reprezentuje znak zapisany w systemie Unicode zapamiętany na dwóch bajtach. W języku Java z typem char związanych jest wiele użytecznych metod np.

```
char b = 'B';
char b1 = (char)66; // Kod litery B
char t = '\t';
char c = (char)0x0042;
System.out.println(b + " " + b1 + t + c);
String s = "abc1234.";
for (int i = 0; i < s.length(); i++)
    System.out.print(s.charAt(i));
System.out.println();
boolean bt1 = Character.isLetter(s.charAt(2));
boolean bt2 = Character.isDigit(s.charAt(5));
boolean bt3 = Character.isSpaceChar(s.charAt(7));
System.out.println(bt1+" "+bt2+" "+bt3);
```

run:

B B B

abc1234.

true true false

Wielkie liczby

Jeżeli precyzja podstawowych typów liczb nie wystarcza można skorzystać z dwóch klas *BigInteger* i *BigDecimal* pakietu *java.math*. Klasa *BigInteger* obsługuje arytmetykę liczb całkowitych, a klasa *BigDecimal* arytmetykę liczb zmiennoprzecinkowych.

Uwaga. Java nie pozwala na przeciążenie operatorów. Dlatego zaprojektowano odpowiednie metody realizujące operacje na liczbach typu *BigInteger* i *BigDecimal*. Projektanci języka Java przeciążyli operator dodawania +, tworząc jego wersję łączenia łańcuchów i nie zezwolili programistą piszącym w języku Java na przeciążanie operatorów arytmetycznych.

java.math.BigInteger

- **BigInteger** add(**BigInteger** *another*) – dodawanie liczb
- **BigInteger** subtract(**BigInteger** *another*) – odejmowanie liczb
- **BigInteger** multiply(**BigInteger** *another*) – mnożenie liczb
- **BigInteger** divide(**BigInteger** *another*) – dzielenie liczb
- **BigInteger** mod(**BigInteger** *another*) – reszta z dzielenia modulo liczb
- **int** compareTo(**BigInteger** *another*) – zwraca zero, jeśli liczby są równe. Wynik ujemny, jeśli dana liczba jest mniejsza niż *another* i wynik dodatni, jeśli dana liczba jest większa od *another*.
- **Static BigInteger** valueOf(**long** *x*) – zwraca wielką liczbę o wartości *x*

Zadanie. Należy napisać program obliczający dwumian Newtona wg. wzoru:

$$\binom{n}{k} = \sum_{i=0}^n \frac{n! (n - k)!}{k!}$$

```
Scanner in = new Scanner(System.in);  
System.out.print("Podaj n: ");  
int n = in.nextInt();  
System.out.print("Podaj k: ");  
int k = in.nextInt();  
// Przeczytanie liczb.
```

```
BigInteger sm = BigInteger.valueOf(1);  
for(int i =1; i <= k; i++)  
    sm = sm.multiply(BigInteger.valueOf(n-i+1))  
        .divide(BigInteger.valueOf(i));  
// Obliczenie dwumianu Newtona.  
System.out.println("C = " + sm);  
// Wypisanie wyniku.
```



run:



Podaj n: 4



Podaj k: 2



C = 6

java.math.BigDecimal

- **BigDecimal(Double val)** – konwertuje liczbę typu double
- **BigDecimal(String val)** – konwertuje łańcuch na liczbę *BigDecimal*
- **BigDecimal add(BigDecimal another)** – dodawanie liczb
- **BigDecimal subtract(BigDecimal another)** – odejmowanie liczb
- **BigDecimal multiply(BigDecimal another)** – mnożenie liczb
- **BigDecimal divide(BigDecimal divisor)** – dzielenie liczb
- **BigDecimal divide(BigDecimal divisor, int roundingMode)** – dzielenie liczb z zaokrągleniem *roundingMode*

java.math.BigDecimal

- **int** compareTo(**BigInteger** *another*) – zwraca zero, jeśli liczby są równe. Wynik ujemny, jeśli dana liczba jest mniejsza niż *another* i wynik dodatni, jeśli dana liczba jest większa od *another*.
- **Static BigDecimal** valueOf(**double** *val*) – zamienia liczbę typu *double* na *BigDecimal*
- **Static BigDecimal** valueOf(**long** *unscaleVal*, **int** *scale*) – zwraca wielką liczbę zmiennoprzecinkową o wartości *unscaleVal* podzielonej przez 10^{scale}

Przykład użycia liczb typu *BigDecimal*

```
String str = "123456789.123456789";  
BigDecimal a= new BigDecimal(str);  
BigDecimal b= new BigDecimal(str);  
BigDecimal c= a.add(b);  
System.out.println("a + b = " + c);  
if(c.compareTo(a)>0)  
    System.out.println(" c > a");  
else  
    System.out.println(" c <= a");
```



run:



a + b = 246913578.246913578



c > a

Konwertowanie łańcuchów do typów numerycznych

Dla podstawowych typów numerycznych jest dostępna metoda konwersji łańcuchów znakowych *parse*. Ze względu na angielski interfejs liczby zmiennoprzecinkowe są wprowadzane z kropką dziesiętną.

```
byte i0 = Byte.parseByte("-12");  
short i1 = Short.parseShort("2013");  
int i2 = Integer.parseInt("2014");  
long i3 = Long.parseLong("2011");  
float d1 = Float.parseFloat("12.1");  
double d2 = Double.parseDouble("1.2e10");
```

```
run:  
-12 2013 2014 2011  
12.1 1.2E10
```

Polonizacja wprowadzanych liczb

Klasa *NumberFormat* jest klasą abstrakcyjną, umożliwiającą formatowanie i interpretację danych wejściowych. Jest ona zwracana przez metodę *getInstance()*, pozwalającą na formatowanie danych wg. stylu lokalnego. Poniżej zamieszczony przykład programu pokazuje wprowadzanie liczb zmiennoprzecinkowych w interfejsie polskim.

```
import java.text.*;
import java.util.*;
public class C2b {
    public static void main(String[] args)
        throws ParseException {
```

```
Locale locale = new Locale("pl","PL");  
NumberFormat pl = NumberFormat.getInstance(locale);  
float a = pl.parse("-1,75").floatValue();  
double b = pl.parse("12,5").doubleValue();  
System.out.println(a + " " + b);  
System.out.printf(locale,"%g %g%n", a, b);  
}  
}
```



run:



-1.75 12.5



-1,75000 12,5000

Formatowanie tekstu

Pakiet *java.text* pozwala na formatowanie i przeszukiwanie tekstu. Klasa *NumberFormat* jest używana do formatowania liczb, a klasa *DateFormat* do formatowania daty i czasu. Klasa *DataFormat* jest zwracana przez metodę *getDateInstance()*, którą możemy wywołać z lokalnym sposobem wyświetlania:

```
static final DateFormat getDateInstance( int style, Locale local)
```

Klasa *SiipleDateFormat* jest podklasą klasy *DateFormat*. Pozwala na zdefiniowanie własnych wzorców formatu używanego do wyświetlania informacji o dacie i czasie. Jeden z konstruktorów ma składnie:

```
SipleDataFormat( String formatString, Locale local)
```

```
import java.text.*;
import java.util.*;
public class C2a {
    public static void main(String[] args) {
        Locale lp = new Locale("pl", "PL");
        Date date = new Date();
        DateFormat df =
            DateFormat.getInstance(DateFormat.SHORT, lp);
        System.out.println(df.format(date));
        df = DateFormat.getInstance(DateFormat.MEDIUM, lp);
        System.out.println(df.format(date));
        df = DateFormat.getInstance(DateFormat.LONG, lp);
        System.out.println(df.format(date));
        df = DateFormat.getInstance(DateFormat.FULL, lp);
        System.out.println(df.format(date));
    }
}
```

```

SimpleDateFormat sdf = new
SimpleDateFormat("hh:mm:ss",lp);
System.out.println(sdf.format(date));
sdf = new
SimpleDateFormat("dd MM yyyy hh:mm:ss zzz",lp);
System.out.println(sdf.format(date));
sdf = new SimpleDateFormat("E MMM dd yyyy",lp);
System.out.println(sdf.format(date));
}
}

```



03:20



03:20:12



03:20:12 CEST



03:20:12 CEST

03:20:12

03 07 2018 03:20:12 CEST

Wt lip 03 2018

Formatowanie daty i czasu - *SipleDateFormat*

Symbole	Opis
a	AM lub PM
d	Dzień miesiąc (1 – 31)
h	Godzina w formacie 12 – godzinnym
k	Godzina w formacie 24 – godzinnym
m	Minuta (0 – 59)
s	Sekunda (0 – 59)
w	Tydzień roku (1 – 52)
y	Rok
z	Strefa czasowa

Symbol	Opis
D	Dzień roku (1 – 366)
E	Dzień tygodnia (np. Thursday)
F	Dzień tygodnia w danym miesiącu
G	Era (AD lub BC)
H	Godzina (0 – 23)
K	Godzina (0 – 11)
M	Miesiąc
S	Milisekunda
W	Tydzień miesiąca (1 – 5)
Z	Strefa czasowa w formacie RFC832

Priorytety operatorów liczbowych

Operator	Opis
()	Nawiasy mówiące o kolejność wykonywanych operacji.
~ ++ --	Dopełnienie bitowa, przedrostkowa lub przyrostkowa inkrementacja albo dekrementacja.
* / %	Mnożenie, dzielenie i reszta z dzielenia modulo.
+ -	Dodawanie i odejmowanie.
<< >>	Przesunięcie bitowe w lewo i przesunięcie bitowe w prawo.
&	Koniunkcja bitowa.
	Alternatywa bitowa.
^	XOR – bitowa alternatywa wykluczająca.

Operatory relacyjne i logiczne

Operator	Opis i przykład
==	Równe <code>if (i == j) ...</code>
!=	Różne <code>if (i != j) ...</code>
<	Mniejszy od <code>if (i < j) ...</code>
<=	Mniejszy równy od <code>if (i <= j) ...</code>
>	Większy od <code>if (i > j) ...</code>
>=	Większy równy od <code>if (i >= j) ...</code>
!	Negacja logiczna <code>if (!(i < j)) ...</code>
&&	Koniunkcja logiczna <code>if (i >= 0 && i < n) ...</code>
	Alternatywa logiczna <code>if (a[i] > 0 b[i] > 0) ...</code>

Klasa *String*

W środowisku Java zdefiniowana została klasa *String* przeznaczana do manipulacji na ciągach znakowych. W celu utworzenia egzemplarza klasy *String* i przypisania mu jakiejś wartości użyjemy następującej instrukcji:

```
String s1 = "Łańcuch znaków";
```

Wszędzie tam gdzie nie wystarcza klasa *String* należy używać klasy *StringBuffer* umożliwiającej manipulowanie długością łańcucha znaków.

```
String s1 = "Łańcuch znaków";  
String s2 = s1;  
s1 += " i dodany łańcuch znaków";
```

Tablice - deklaracja

Tablica jest indeksowaną kolekcją elementów tego samego typu.

Składnia:

Type [] identyfikator = {kolekcja elementów};

albo

Type [] identyfikator = new Type[wymiar];

Przykład:

```
int a[] = { 1, 2, 3 };  
for(int b: a)  
    System.out.println(b);  
for (int i = 0; i < a.length; i++)  
    System.out.println(a[i]);  
int c[] = new int[3];
```

Zdanie. Należy napisać program wczytując z klawiatury n liczb całkowitych do tablicy i wyszukujący w tablicy element największy i pozycję, na której został umieszczony.

```
Scanner in = new Scanner(System.in);
System.out.print("Podaj n = ");
int n = in.nextInt();

int a[] = new int[n];
for(int i=0; i < a.length; i++){
    System.out.printf("a[%d] = ", i);
    a[i] = in.nextInt();
}
int max = a[0];
int t = 0;
for (int i = 1; i < a.length; i++)
    if (a[i] > max) { max = a[i]; t = i; }
System.out.printf("max=a[%d] = %d\n",t,a[t]);
```

```
run:
Podaj n = 5
a[0] = 3
a[1] = -8
a[2] = 67
a[3] = 4
a[4] = 5
max=a[2] = 67
```

Zadanie. Należy napisać metodę w klasie Algorytm wyszukującą indeks największego elementu w tablicy.

```
package c2e;
import java.util.*;
class Algorytm{
    public int max(int a[]) {
        int max = a[0];
        int t = 0;
        for(int i=1; i< a.length; i++)
            if(a[i] > max)
                { max = a[i]; t = i;}
        return t;
    }
}
```

```
public class C2e {  
    public static void main(String[] args) {  
        Scanner in = new Scanner(System.in);  
        System.out.print("Podaj n = ");  
        int n = in.nextInt();  
        int a[] = new int[n];  
        for(int i=0; i < a.length; i++){  
            System.out.printf("a[%d] = ", i);  
            a[i] = in.nextInt();  
        }  
        Algorytm at = new Algorytm();  
        int t = at.max(a);  
        System.out.printf("max=a[%d] = %d%n", t, a[t]);  
    }  
}
```

run:
Podaj n = 3
a[0] = 2
a[1] = -1
a[2] = 1
max=a[0] = 2

Tablice postrzępione

Elementy tablicy mogą być kolejnymi tablicami. Tak zagnieżdżone tablice nazywamy tablicami postrzępionymi (ang. jagged arrays). Popatrzmy na konstrukcję:

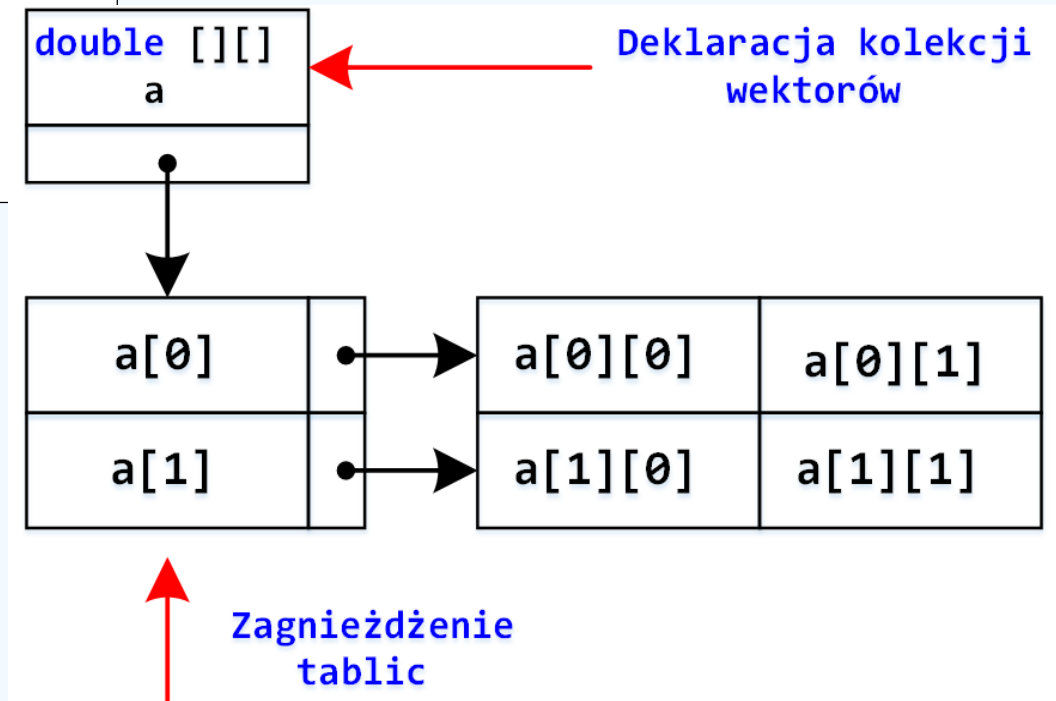
```
double [][] a= new double[2][2];
```

```
double [][] a= new double[2][];
```

```
    a[0]= new double[2];
```

```
    a[1]= new double[2];
```

Utworzona tablica jest macierzą kwadratową zawierającą cztery elementy. Oznaczenia wierszy i kolumn zostały przedstawione na rysunku.



Zadanie. Należy napisać program znajdujący rozwiązanie układu równań liniowych o dwóch niewiadomych.

```
Scanner in = new Scanner(System.in);
double a[][] = new double[2][2];
// Deklaracja macierzy A.
double [] b = new double[2];
// Deklaracja wektora b.
for (int i = 0; i < 2; i++)
{
    for (int j = 0; j < 2; j++) {
        System.out.printf("Podaż a[%d][%d]:", i, j);
        a[i][j] = in.nextDouble(); }
    // Wczytanie elementów macierzy A.
    System.out.printf("Podaż b[%d]:", i);
    // Wczytanie elementów wektora b.
    b[i] = in.nextDouble();
}
```

```
double W = a[0][0] * a[1][1] - a[0][1] * a[1][0];
double Wx = b[0] * a[1][1] - a[0][1] * b[1];
double Wy = a[0][0] * b[1] - b[0] * a[1][0];
//Wyznaczenie wyznaczników układu
if (W != 0){
    //Sprawdzenie kolejnych warunków rozwiązalności układu.
    System.out.printf("x[0]=%g%n", Wx / W);
    System.out.printf("x[1]=%g%n", Wy / W);}
else if (Wx != 0)
    System.out.println("Układ równań jest sprzeczny");
else
    System.out.println("Układ równań jest nieoznaczony");
```



```
run:
Podaj a[0][0]:2
Podaj a[0][1]:1
Podaj b[0]:3
Podaj a[1][0]:1
Podaj a[1][1]:2
Podaj b[1]:3
x[0]=1,00000
x[1]=1,00000
```

Tablica stringów jest dwuwymiarową postrzępioną tablicą znaków Unicode

```
String [] miesiace =  
{ "Styczeń", "Luty", "Marzec",  
  "Kwiecień", "Maj", "Czerwiec", "Lipiec",  
  "Sierpień", "Wrześcień", "Październik",  
  "Listopad", "Grudzień" };  
// Deklaracja tablicy miesięcy.  
for(String s: miesiace)  
    System.out.println(s);  
    // Wypisanie miesięcy.  
char c = miesiace[1].charAt(1);  
// Zapisanie znaku Unicode  
// znajdującego się  
// na pozycji [1][1].  
System.out.printf("Znak na pozycji [1][1]:%c%n", c);
```

run:
Styczeń
Luty
Marzec
Kwiecień
Maj
Czerwiec
Lipiec
Sierpień
Wrześcień
Październik
Listopad
Grudzień
Znak na pozycji [1][1]:u

Wybrane składowe klasy Arrays

Klasa Arrays definiuje szereg właściwości metod pomocnych w modyfikowaniu zawartości tablic i wyszukiwaniu potrzebnych informacji. Jedną z bardziej istotnych metod jest metoda sortowania dostępna w wielu przeciążonych wersjach. Najprostsza wersja metoda pobiera jednowymiarową tablicę i sortuje jej elementy. Rozbudowane wersje umożliwiają sortowanie tablic za pomocą interfejsu zdefiniowanego przez użytkownika.

```
Locale lp = new Locale("pl","Pl");  
Collator col = Collator.getInstance(lp);  
String [] tablica = {"ć","a", "a","c"};  
Arrays.sort(tablica,col);  
for(String x: tablica)  
    System.out.println(x);
```

run:
a
a
c
ć

Metoda *split()*

Istnieje możliwość podzielenia sekwencji wejściowej na fragmenty. Składnia:

String[] split(**Charsequence** *str*)

Przykładowo poniższy program dzieli na wyrazy ciąg znaków, w którym są one rozdzielone znakami spacji, przecinka, kropki i wykrzyknika.

```
String s = "jeden dwa. alfa 12!zrobione.";
String strs[] = s.split("[ .,!]");
for(String x: strs)
    System.out.println("Wyraz:" + x);
```

run:

Wyraz:jeden

Wyraz:dwa

Wyraz:

Wyraz:alfa

Wyraz:12

Wyraz:zrobione

Zaradnie. W łańcuchu zapisane są dwie liczby rzeczywiste oddzielone spacjami zapisane w notacji polskiej. Należy obliczyć sumę tych liczb i wydrukować w notacji polskiej.

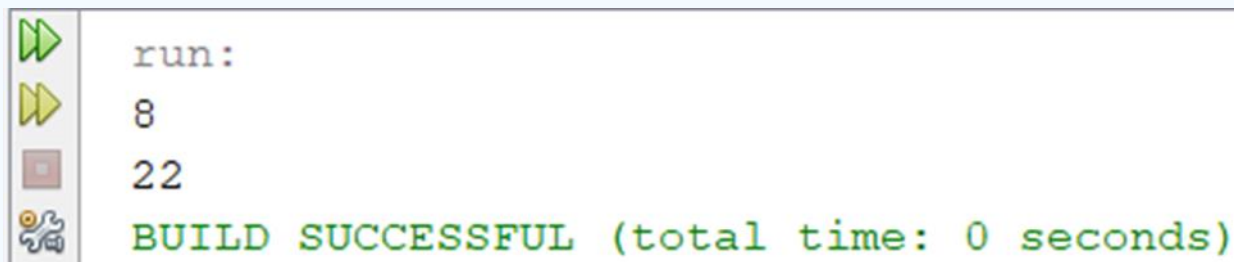
```
import java.text.*;
import java.util.*;
public class C2j {
    public static void main(String[] args)
        throws ParseException {
        Locale lp = new Locale("pl", "PL");
        NumberFormat pl = NumberFormat.getInstance(lp);
        String st = " 1,2  6,89";
        String tab[] = st.trim().split("\\s");
        double d1 = pl.parse(tab[0]).doubleValue();
        double d2 = pl.parse(tab[tab.length-1]).doubleValue();
        System.out.printf(lp, "Suma = %g%n", d1 + d2);
    }
}
```

run:
Suma = 8,09000

Przeszukiwanie łańcuchów znakowych

Czasami jest konieczne wyszukiwanie w ciągu znaków określonego zestawu znaków. W tym celu można wykorzystać kilka różnych metod. Aby wyszukać w ciągu znaków pewne słowo i uzyskać jego pozycję w tym ciągu, możemy użyć metody `IndexOf()` lub `LastIndexOf()`. Metoda `IndexOf()` zwraca pozycję pierwszego wystąpienia określonego słowa w ciągu znaków, natomiast metoda `LastIndexOf()` zwraca pozycję ostatniego wystąpienia tego słowa.

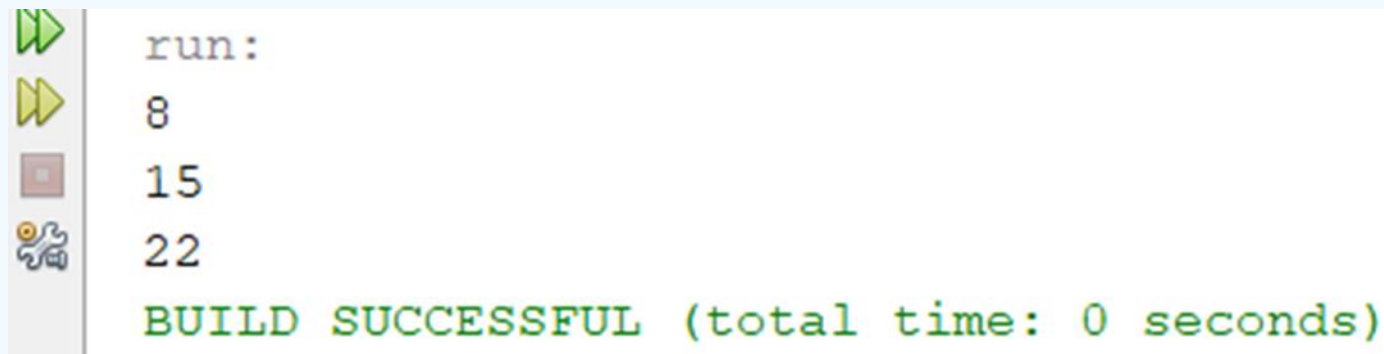
```
String st =  
    "To jest długi, długi, długi ciąg znaków...";  
System.out.println(st.indexOf("długi"));  
System.out.println(st.lastIndexOf("długi"));
```



run:
8
22
BUILD SUCCESSFUL (total time: 0 seconds)

Gdybyśmy chcieli wyszukać wszystkie wystąpienia danego słowa, możemy użyć metody Indexof() w ramach pętli:

```
int position = -1;
do
{
    position = st.indexOf("długi", ++position);
    if (position >= 0)
        System.out.println(position);
} while (position >= 0);
```



```
run:
8
15
22
BUILD SUCCESSFUL (total time: 0 seconds)
```